IMAGE PROCESSING · ปีการศึกษา 2569

Image Genny

เว็บแอปพลิเคชันสร้างและแก้ไขภาพด้วย AI

สาขาวิศวกรรมคอมพิวเตอร์ คณะเทคโนโลยีดิจิทัล สถาบันเทคโนโลยีจิตรลดา

อาจารย์ผู้สอน: อาจารย์กฤษฎา พรหมสุทธิรักษ์

ทีม

สมาชิก 4 คน แบ่งกันคนละส่วนของระบบ

ชื่อ	รหัสนักศึกษา	ส่วนที่รับผิดชอบ
นายพิพรรธน์ คล้ายสุข	6710301003	AI Service (เชื่อมต่อโมเดลสร้างภาพ และระบบคิว)
นางสาวสุนันทา บุญปกครอง	6710301019	Database
นายอาเบต บัวดอกตูม	6710301027	Backend (Flask API)
นายสิรภพ เสงี่ยมศักดิ์	6710301036	Frontend (หน้าเว็บ)

ระบบนี้ทำอะไร

พิมพ์คำอธิบาย → ได้ภาพที่ AI สร้างให้

โมเดล Stable Diffusion **รันบนเครื่องของทีมเอง** ไม่ได้ใช้บริการภายนอก

1 · สมาชิก

สมัครสมาชิกและเข้าสู่ระบบ

2 · สร้างภาพ

Text-to-Image เลือกสไตล์และปรับค่าได้

3 · แก้ไขภาพ

ลบบางส่วน ครอป ปรับแสงสี หมุนและพลิก

4 · คลังภาพ

เก็บภาพส่วนตัว ดาวน์โหลด PNG

5 · admin

ระงับและปลดระงับบัญชีผู้ใช้

สถาปัตยกรรม

4 ส่วนรันแยกเครื่องได้ คุยกันผ่าน REST API

Frontend + nginx
:80 · หน้าเว็บ
ส่งต่อ /api/ ไป Backend

→

Backend
Flask :5000
ตรวจ login บันทึกงาน

→

AI Service
FastAPI :8001
ตรวจข้อมูล จัดคิว

→

Forge Neo
:7860 · Stable Diffusion
ใช้ GPU เจนภาพ

↓ Database
PostgreSQL · เข้าถึงผ่าน Backend เท่านั้น

ระบบแบบ **distributed** คุยกันด้วย **HTTP + JSON**

หน้าเว็บคุยกับ **Backend เท่านั้น**

Backend รู้ว่าผู้ใช้เป็นใคร · **AI Service ไม่รู้**

เทคโนโลยี

เทคโนโลยีที่ใช้ในแต่ละส่วน

ส่วน	เทคโนโลยี
Frontend	HTML, CSS, JavaScript (ไม่มี framework), MediaPipe Interactive Segmenter
Reverse proxy	nginx
Backend	Python, Flask, Flask-SQLAlchemy, Flask-JWT-Extended, Flask-Cors
AI Service	Python, FastAPI, Uvicorn, Pydantic, httpx, asyncio
โมเดลสร้างภาพ	Stable Diffusion ผ่าน Forge Neo (ติดตั้งด้วย Stability Matrix)
Database	PostgreSQL
ทำงานร่วมกัน	Git และ GitHub (แยก branch แล้วเปิด Pull Request)

การใช้งาน

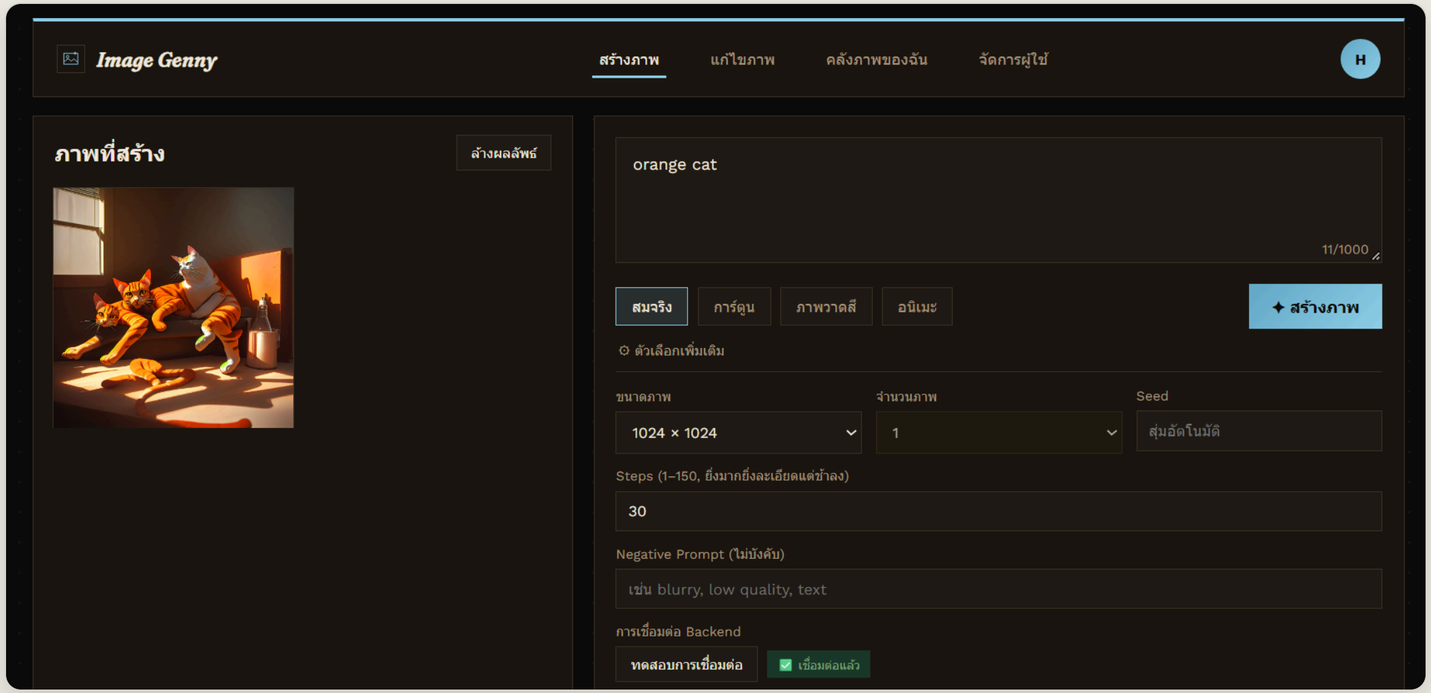
เข้าสู่ระบบด้วย token (JWT)

1. สมัครสมาชิกด้วยชื่อผู้ใช้และรหัสผ่าน (อย่างน้อย 6 ตัวอักษร)
2. เข้าสู่ระบบ → ระบบออก **token (JWT) อายุ 24 ชั่วโมง**
3. หน้าเว็บเก็บ token ไว้ใน browser และแนบไปกับทุกคำขอ
4. เปิดหน้าหลักโดยไม่ได้ login → ถูกพากลับหน้า login

> ความปลอดภัย: รหัสผ่านถูกเก็บเป็น **hash** ไม่ได้เก็บตัวรหัสผ่าน

การใช้งาน

สร้างภาพ: พิมพ์ → เลือกสไตล์ → กดสร้าง



สไตล์ สมจริง · การ์ตูน · ภาพวาดสี · อนิเมะ — ต่อข้อความท้าย prompt

ขนาด 1024×1024, 1024×768, 768×1024, 1344×768

Steps 1–150 (เริ่มที่ 30)

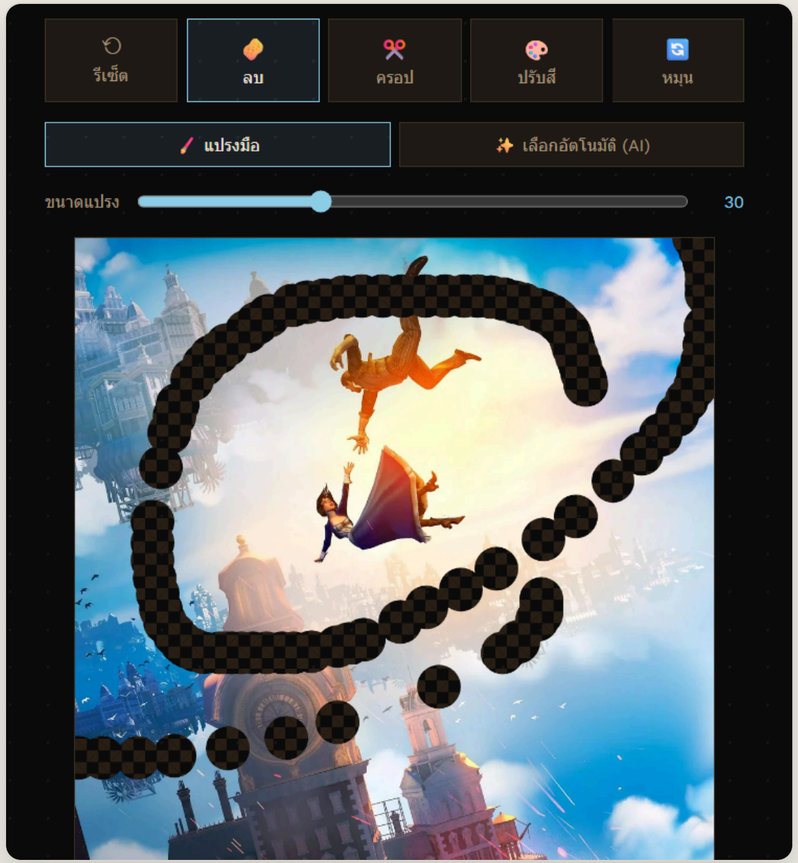
Seed เลขเดิม + prompt เดิม = ภาพเดิม

Negative Prompt สิ่งที่ไม่อยากให้มี

เจนครั้งละ 1 ภาพ · โมเดลคือตัวที่เลือกไว้ใน Forge Neo (เลือกจากหน้าเว็บไม่ได้)

การใช้งาน

แก้ไขภาพ: ประมวลผลใน browser ทั้งหมด

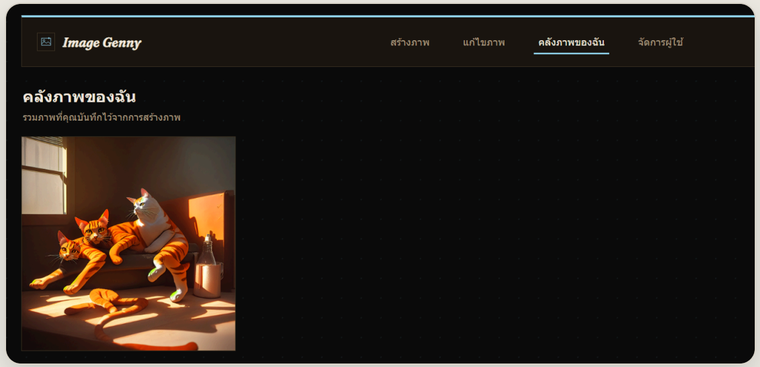


ลบ	แปรงระบาย หรือ "เลือกอัตโนมัติ (AI)" ส่วนที่ลบเป็นพื้นโปร่งใส
ครอป	ลากเลือกพื้นที่
ปรับสี	สว่าง คอนทราสต์ อิ่มสี ขาวดำ ซีเปีย
หมุน	หมุน 90° และพลิก
ย้อนกลับ	สูงสุด 15 ขั้น · รีเซ็ตกลับต้นฉบับ

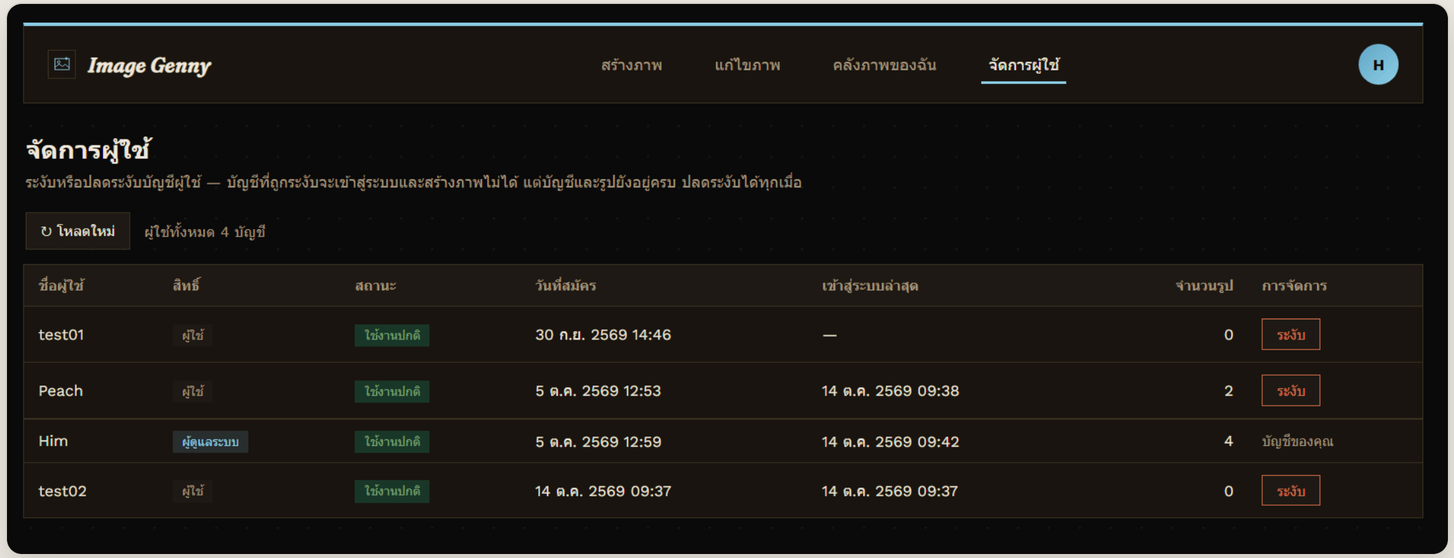
ไม่ส่งภาพไป server · "เลือกอัตโนมัติ (AI)" ใช้ **MediaPipe Interactive Segmenter** ในเครื่องผู้ใช้ · ดาวน์โหลด PNG

การใช้งาน

คลังภาพ และผู้ดูแลระบบ



คลังภาพ กดหัวใจเก็บ กดซ้ำเอาออก ดาวน์โหลดได้

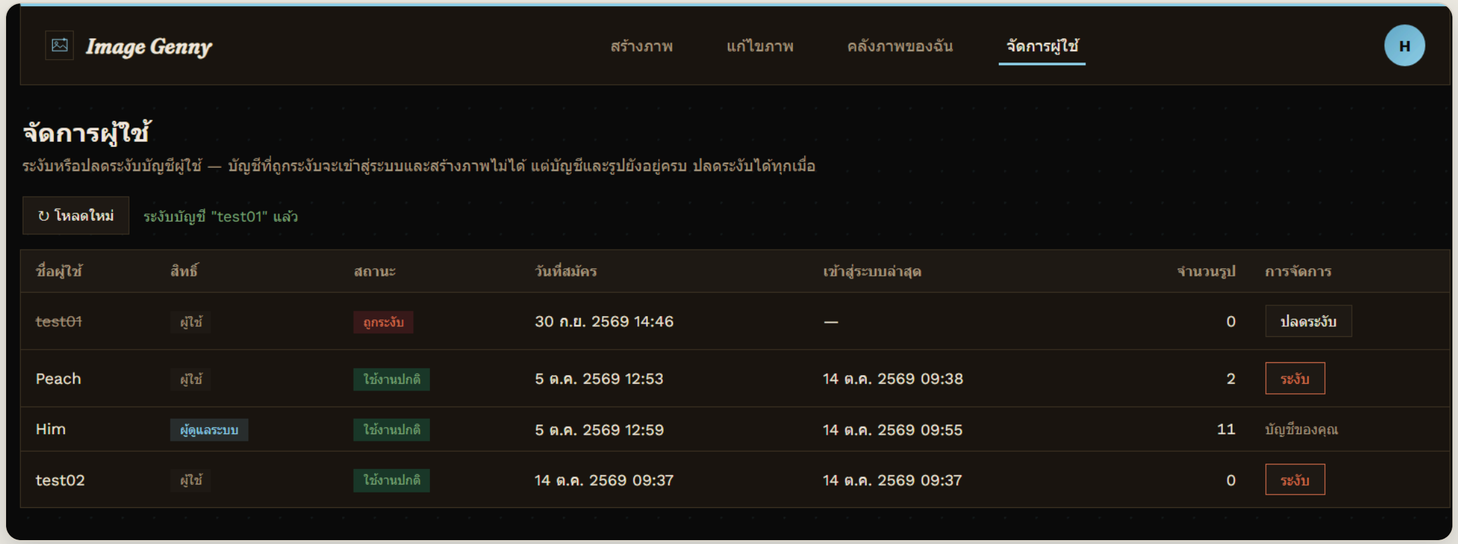


- **admin** เห็นเมนู "จัดการผู้ใช้": ชื่อ สิทธิ์ สถานะ วันสมัคร login ล่าสุด จำนวนรูป
- ระงับแล้ว**ใช้งานไม่ได้ทันที** แต่ข้อมูลและรูปไม่ถูกลบ
- ระงับตัวเองหรือ admin คนอื่นไม่ได้

การใช้งาน

admin ระงับบัญชี → บัญชีนั้นเข้าระบบไม่ได้ทันที

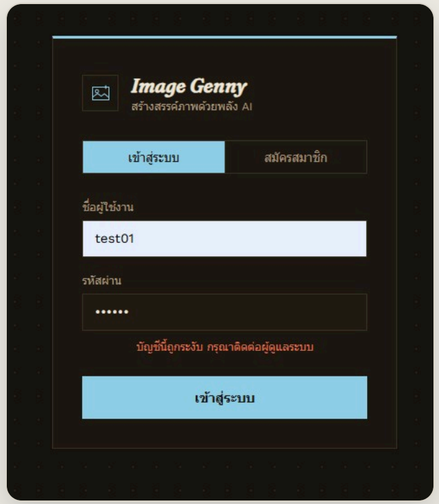
1 · admin กด "ระงับ" ที่บัญชี test01



สถานะเปลี่ยนเป็น **ถูกระงับ** ชื่อขีดฆ่า ปุ่มเปลี่ยนเป็น "ปลดระงับ" · บัญชีและรูปยังอยู่ครบ

→

2 · test01 พยายาม login



ได้ข้อความ "บัญชีนี้ถูกระงับ กรุณาติดต่อผู้ดูแลระบบ"

> Backend ตรวจสถานะบัญชีจากฐานข้อมูล**ในทุกคำขอ** — token เดิมของ test01 ก็ใช้ไม่ได้ทันที ไม่ต้องรอหมดอายุ

แต่ละส่วนทำงานอย่างไร

Frontend: 3 ไฟล์ ไม่มี framework

`login.html`

หน้าเข้าสู่ระบบและสมัครสมาชิก

`index.html`

4 หน้าย่อยในไฟล์เดียว สลับด้วยการซ่อน/แสดง: สร้างภาพ, แก้ไขภาพ, คลังภาพ, จัดการผู้ใช้

`config.js`

ที่อยู่ของ Backend (BACKEND_URL)

- ทุกคำขอแนบ header `Authorization: Bearer <token>`
- ทุกครั้งที่เปิดหน้า ถาม Backend ว่าบัญชียังใช้ได้ไหม และมีสิทธิ์อะไร
- เมนู admin ซ่อนจากผู้ใช้ทั่วไป แต่**สิทธิ์จริงตรวจที่ Backend ทุกครั้ง**

แต่ละส่วนทำงานอย่างไร

Backend: Flask API 10 endpoint ใน 4 กลุ่ม

กลุ่ม	Endpoint
ตรวจสอบ	`GET /api/health`
สมาชิก	`POST /api/auth/register · POST /api/auth/login · GET /api/auth/me`
รูป	`POST /api/image/generate · GET /api/image/history · GET /api/image/<id>`
admin	`GET /api/admin/users · POST .../<id>/suspend · POST .../<id>/unsuspend`

JWT และตรวจสถานะบัญชีจากฐานข้อมูลในทุกคำขอ

บันทึกทุกงาน processing → completed / failed

ส่งงานต่อให้ AI Service แล้ว**รอได้นานสุด 600 วินาที**

แต่ละคน**เห็นเฉพาะงานของตัวเอง**

แต่ละส่วนทำงานอย่างไร

AI Service: ตัวกลางระหว่าง Backend กับ Forge Neo

FastAPI · 5 ไฟล์ · 331 บรรทัด

- `POST /generate` สร้างรูป คืน base64
- `GET /health` ต่อ Forge ได้ไหม
- `GET /` service รันอยู่ไหม

ซ่อน Forge

Backend รู้จักแค่ /generate

ตรวจก่อน

ค่าผิดถูกปฏิเสธด้วย 422 ก่อนถึง GPU

จัดคิว

GPU ทำได้ทีละภาพ

แปลง error

เป็นข้อความที่ผู้ใช้อ่านได้

> AI Service **ไม่มีโมเดล AI ในตัว** — รับ ตรวจ จัดคิว แล้วส่งต่อให้ Forge Neo เป็นคนเจนภาพ (หน้าถัดไป)

แต่ละส่วนทำงานอย่างไร

คำขอเดินผ่าน 3 ไฟล์ แล้วส่งให้ Forge Neo เจนภาพ

1	`main.py`	รับ `POST /generate` จาก Backend · Pydantic ตรวจข้อมูลตามแบบฟอร์มใน `models.py` (ผิด = 422)
2	`queue_manager.py`	`enqueue_generate()` เช็คคิวเต็มไหม แล้วรอจนถึงตาตัวเอง (Semaphore)
3	`forge_client.py`	`generate_image()` **จุดสุดท้ายใน AI Service** — ส่ง `POST /sdapi/v1/txt2img` ไป Forge แล้วรอ
4	**Forge Neo :7860**	**โมเดล Stable Diffusion อยู่ที่นี่** ใช้ GPU เจนภาพ แล้วตอบกลับเป็น base64

ขากลับ: forge_client.py → queue_manager.py → main.py → ส่งภาพ base64 กลับ Backend

forge_client.py เป็นไฟล์เดียวที่คุยกับ Forge Neo · ค่าตั้งค่าทั้งหมด (ที่อยู่ Forge, timeout, ขนาดคิว) มาจาก `config.py`

ระบบคิว

เจนทีละภาพ รอได้ 3 งาน เกินนั้นตอบทันที

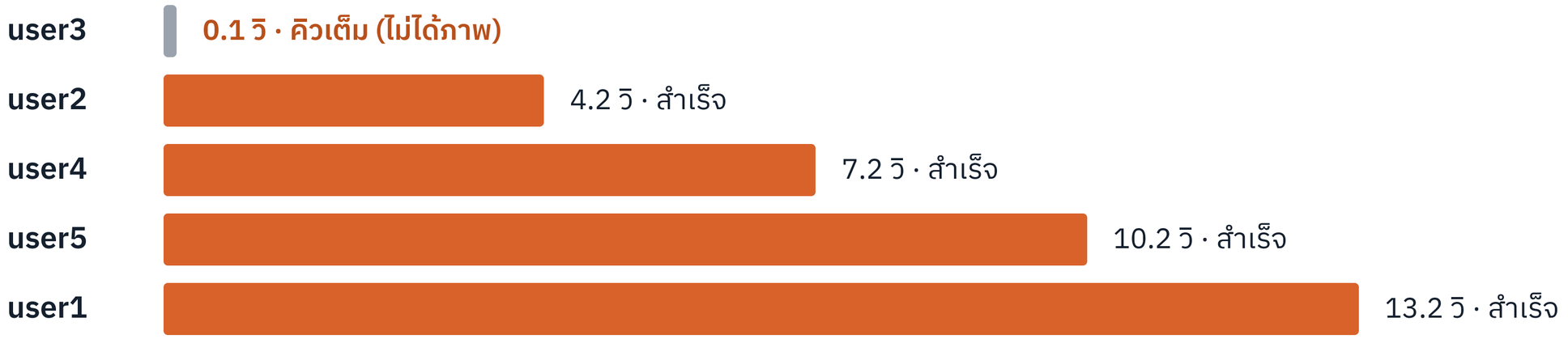
คำขอที่ 1	ที่ 2	ที่ 3	ที่ 4	ที่ 5+
เจนทันที	รอคิว	รอคิว	รอคิว	"คิวเต็ม" ทันที

- `asyncio.Semaphore(1)` ให้เจน**ทีละ 1 ภาพ** งานอื่นรอตามลำดับที่มาถึง (FIFO)
- รอคิวได้สูงสุด **3 งาน** (ตั้งค่าได้) ไม่ปล่อยให้รอจนหมดเวลา
- ทำงานแบบ **asynchronous** ระหว่างรอ Forge ยังรับคำขออื่นได้ เช่น เช็คสถานะ

Forge Neo มีคิวของตัวเองอยู่แล้ว ชั้นนี้เพิ่ม: จำกัดจำนวนงานที่รอ · ปฏิเสธทันที · บันทึก log

ผลทดสอบคิว

5 คำขอพร้อมกัน: เจน 4 ภาพตามคิว ปฏิเสธ 1



เวลาที่ได้รับคำตอบ (วินาที) · ส่งห่างกันไม่เกิน 0.9 มิลลิวินาที · ภาพ 768×768, 8 steps · ทดสอบบนเครื่องเดียว

เสร็จห่างกัน**ภาพละ 3 วิ** = เจนทีละภาพ ไม่ซ้อน

ได้ 4 ภาพต่างกัน แต่ละคน**เห็นเฉพาะงานตัวเอง**

ลำดับขึ้นกับ**คำขอใครมาถึงก่อน**

Database

PostgreSQL 2 ตาราง เชื่อมกันด้วย user_id

users

บัญชีผู้ใช้

- username, password_hash
- role (user / admin)
- status (active / suspended)
- last_login

← **foreign key** `user_id → users.id`

image_tasks

1 แถวต่อการกดสร้าง 1 ครั้ง

- user_id, prompt
- status
- image_url (ภาพเป็น base64)
- created_at

ย้ายจาก SQLite มาใช้ **PostgreSQL** แล้ว ด้วยการแก้ค่าตั้งค่า (DATABASE_URL) บรรทัดเดียว ไม่ต้องแก้โค้ด — เพื่อให้ทุกเครื่องใช้ข้อมูลชุดเดียวกัน

เส้นทางการทำงาน

เกิดอะไรขึ้นเมื่อกด "สร้างภาพ"

1	**Frontend**	รวม prompt กับข้อความสไตล์ ส่งไป Backend พร้อม token
2	**Backend**	ตรวจ token และสถานะบัญชี
3	**Backend → Database**	บันทึกงานใหม่ สถานะ processing
4	**Backend → AI Service**	ส่งคำขอสร้างรูป แล้วรอ
5	**AI Service**	ตรวจข้อมูล และต่อคิว
6	**AI Service → Forge Neo**	Forge Neo ใช้ GPU เจนภาพ ตอบกลับเป็น base64
7	**AI Service → Backend**	ส่งภาพกลับ
8	**Backend → Database**	เก็บภาพ สถานะ completed
9	**Backend → Frontend**	ส่งภาพกลับ หน้าเว็บแสดงผล

การจัดการข้อผิดพลาด

ทุกปัญหาส่งถึงผู้ใช้เป็นข้อความที่บอกสาเหตุ

สถานการณ์	สิ่งที่ผู้ใช้เห็น
มีคนรอคิวครบแล้ว	คิวเต็ม (มีงานรออยู่ 3 งาน) กรุณาลองใหม่ภายหลัง
Forge Neo ไม่ได้เปิด	เชื่อมต่อ Forge Neo ไม่ได้
เจนนานเกินกำหนด	Forge Neo ไม่ตอบสนองภายใน 300 วินาที
บัญชีถูกระงับ	พากลับหน้า login พร้อมข้อความ "บัญชีนี้ถูกระงับ"
token หมดอายุ	พากลับหน้า login

งานที่ไม่สำเร็จถูกบันทึกเป็น **failed**

timeout เป็นชั้น: AI Service รอ Forge **300 วิ**ต่อภาพ · Backend และ nginx รอ **600 วิ** รวมเวลารอคิว

การทดสอบ

ทดสอบผ่านทั้งเครื่องเดียวและข้าม 3 เครื่อง

การทดสอบ	ผล
ทดสอบทั้งเส้นทางผ่านหน้าเว็บจริงด้วย browser อัตโนมัติ 16 ขั้นตอน (สมัคร, login, สร้างภาพ, ประวัติ, admin ระงับ/ปลดระงับ, บัญชีที่ถูกระงับใช้งานไม่ได้)	**ผ่าน 16/16**
สร้างภาพ 1024×1024 ผ่านหน้าเว็บ ด้วย Forge Neo จริง	**สำเร็จ 18.7 วินาที**
ส่ง 5 คำขอพร้อมกัน	เจน 4 ภาพตามคิว ปฏิเสธ 1
เชื่อมต่อ 3 เครื่องในวงเดียวกัน (Frontend, Backend, AI Service คนละเครื่อง)	สร้างภาพได้
ใช้ nginx เป็น reverse proxy	เปิดหน้าเว็บและเรียก API ได้

3 แถวแรกทดสอบบนเครื่องเดียว · เวลาที่วัดได้ขึ้นกับ GPU ของเครื่องที่ใช้ทดสอบ

ข้อจำกัด

ข้อจำกัดของระบบตอนนี้

ข้อจำกัด	รายละเอียด
เลือกโมเดลจากหน้าเว็บไม่ได้	ใช้โมเดลที่เปิดอยู่ใน Forge Neo
ทำได้เฉพาะ Text-to-Image	ยังไม่มีการแก้ภาพด้วย AI ฝั่ง server
เจนทีละภาพ	GPU ทำได้ทีละงาน ผู้ใช้อื่นต้องรอคิว
ไม่แสดงความคืบหน้า	ไม่บอกลำดับคิวระหว่างรอ
คลังภาพไม่ถาวร	รีเฟรชหน้าแล้วหาย
ภาพเก็บเป็น base64 ใน DB	ฐานข้อมูลโตเร็ว (ราว 0.7–2 MB ต่อภาพ)
AI Service ไม่ตรวจ login	ต้องจำกัดการเข้าถึงด้วย firewall

แนวทางพัฒนาต่อ

สิ่งที่จะทำต่อ

เลือกโมเดลจากหน้าเว็บ

Forge Neo มี API รองรับอยู่แล้ว

แก้ภาพด้วย AI

image-to-image, inpainting, ขยายภาพ

คลังภาพและประวัติจาก DB

รีเฟรชแล้วไม่หาย

แสดงคิวและความคืบหน้า

ผู้ใช้รู้ว่ารออีกนานแค่ไหน

เก็บภาพเป็นไฟล์

แทน base64 ในฐานข้อมูล

สรุป

1 · Text-to-Image

เว็บแอปสร้างภาพจากข้อความ รันโมเดลบนเครื่องของทีมเอง

2 · 4 ส่วน REST API

Frontend, Backend, AI Service, Database

3 · JWT · คิว · admin

ระบบสมาชิก, คิวสำหรับ GPU และผู้ดูแลระบบ

4 · ทดสอบแล้ว

ทั้งบนเครื่องเดียว และข้าม 3 เครื่อง

ถาม-ตอบ / สาธิต

เข้าสู่ระบบ → สร้างภาพ 1 ภาพ → แก้ไขภาพ → admin ระงับบัญชี → บัญชีที่ถูกระงับเข้าไม่ได้